

Lab 3 – Orchestrating Data Pipelines with Apache Airflow

Big Data Processing – EPF Engineering School

Théo Burdinat `theo.burdinat1@epf.edu.fr`

May 20, 2026 | Duration: 3 h | Pair work | Local machine

The Scenario

At the end of the previous lab you delivered a perfectly tuned PySpark job to process **NYC Yellow Taxi** revenue. Your manager loved it. So much, in fact, that they now want that job to run **every single night** for the next year.

A week later, your phone rings at 8 AM:

“The dashboard is empty. What happened?”

You check the server: yesterday’s input file arrived two hours late, your **cron** job ran on an empty folder, crashed silently, and nobody noticed until the CEO opened the dashboard.

Your Mission

Stop firefighting. Replace your fragile **cron** setup with a real orchestrator. You are going to build, step by step, a **production-grade Airflow pipeline** that:

1. Waits for the input file to actually arrive (no more crashes on empty folders).
2. Processes data *partitioned by day* so it can be safely re-run.
3. Can backfill weeks of missed history with a single command.
4. Retries transient failures automatically and alerts you when something is really broken.
5. Stays maintainable: separation of concerns, observability, no anti-patterns.

Working Environment

This lab runs **entirely on your laptop**. No remote cluster, no MinIO server. You will deploy a full Airflow stack locally using Docker Compose.

Evaluation – Live Checkpoints

This lab is graded **in the live session**, in the room, during the session. There is **no written report** to hand in.

Five times during the lab you will reach a **checkpoint**. Each time, you **call the instructor**. The instructor will verify your work and grade it on the spot, based on:

- Correctness of the output (DAG runs, files produced, logs).
- Evidence shown directly in the Airflow UI or the metadata DB.
- Your ability to *explain* what you did and why – not just to show that it works.

Info

The **full grading rubric** for each checkpoint is embedded in the purple **STOP** boxes below (one per part). Each checkpoint contains the evaluated criterias.

Important

If you run out of time, call the instructor with what you have. A partially complete checkpoint is better than a missing one.

Time Budget

Part	Duration	Focus	CP
0 – Setup & Architecture	~30 min	Stand up Airflow; map components to CM3; Pools	CP1
1 – First DAGs	~60 min	Operators, TaskFlow, fan-out/fan-in, TaskGroups	CP2
2 – Sensors & Ingestion	~45 min	FileSensor, validation, sensor modes, stability	CP3
3 – Scheduling & Idempotence	~45 min	<code>execution_date</code> , backfill, catchup safety	CP4
4 – Resilience (if time allows)	~45 min	Broken DAG hunt; retries; SLAs; callbacks	CP5
<i>Bonus parts</i>			
5 – Modern Patterns	–	TaskGroups, branching, Datasets (event-driven)	–
6 – Bonus	–	Dynamic mapping, Variables, TriggerRules, tests	–

Info

The first five parts (CP1–CP5) form the **core** 3 h lab and are graded live. Parts 5 and 6 are deliberately oversized: they exist so that fast pairs have nice material to work on while the rest of the room finishes CP4 or CP5. **You are not expected to finish everything - just do your best, you will get the correction at the end of the day on Moodle.**

Part 0: Setup & Architecture Hunt

Important

Before you start. You should have followed the `Setup_TP3.md` document before coming to the lab. Make sure Docker Desktop is *running* and that the `lab3_student` folder has been extracted somewhere with a short path (e.g. `C:\dev\lab3` on Windows). Long paths or paths with spaces are harder to manage with Docker.

Q0.1 – Stand up the stack

Open a terminal in the `lab3_student` directory and run:

```
docker compose up airflow-init # one-time DB init
docker compose up -d # start all services in background
```

If you already ran the commands before, just run `docker compose up -d` to start the services.

Wait **60–90 seconds**, then open `http://localhost:8080`. Log in with `airflow / airflow`.

Hint

If port 8080 is already in use on your machine, edit the `AIRFLOW_UI_PORT` variable in the `.env` file. Run `docker compose down && docker compose up -d` to apply.

Q0.2 – Component hunt

In a second terminal, run:

```
docker compose ps
```

You should see several containers running. **Fill in the table below:** for each running container, identify which of the four architectural roles from the CM3 it plays, and one observable signal that proves your guess (a port, a log line, a process name).

Container name	CM3 role	Evidence (port, log, process)
airflow-scheduler	...	...
airflow-webserver	...	...
airflow-worker	...	...
airflow-triggerer	...	...
postgres	...	...
redis	...	...

Hint

```
docker compose logs -tail=30 <service> is your friend.
```

Q0.3 – Read the metadata DB

The metadata database is the source of truth. Connect to it from a terminal:

```
docker compose exec postgres psql -U airflow -d airflow
```

Run `\dt` to list tables. Identify the tables that store, respectively:

- DAG definitions parsed by the scheduler.
- Individual task attempts (with their state: running / success / failed).
- DAG run instances (one row per logical run).
- Connections and Variables (encrypted secrets).

Keep the four table names handy – the instructor will ask for them at CP1.

Q0.4 – Architecture reasoning

For each scenario below: **write your prediction first**, then run the exact commands (from the `lab3_student` directory, stack already up `-d`). Fill the table with what you actually observe.

Hint

Unpause `example_hello` in the UI before this exercise so triggers are not ignored while the DAG is paused.

Scenario A — webserver down, trigger via CLI

- Stop the webserver:

```
docker compose stop airflow-webserver
```

- Trigger a run (scheduler and worker still up; CLI runs inside the scheduler container):

```
docker compose exec airflow-scheduler airflow dags trigger example_hello
```

- Watch progress without the browser: either wait ~ 30 s and inspect logs, or run:

```
docker compose exec airflow-scheduler airflow dags list-runs -d example_hello --
no-backfill -o table
```

- (d) Bring the webserver back:

```
docker compose start airflow-webserver
```

Scenario B — scheduler down, trigger from the UI

- (a) Stop the scheduler:

```
docker compose stop airflow-scheduler
```

- (b) In the browser: open the `example_hello` DAG and click **Trigger DAG**. Wait ~60 s. Optionally confirm in the DB shell (from Q0.3) whether a new `dag_run` row appeared and whether `task_instance` states move past `queued`.
- (c) Restart the scheduler:

```
docker compose start airflow-scheduler
```

Scenario C — metadata DB down, UI impact

- (a) With webserver still up, stop Postgres:

```
docker compose stop postgres
```

- (b) Reload `http://localhost:8080` in the browser. Describe what you see (HTTP error, blank page, Airflow error banner, etc.).
- (c) Optional: `docker compose logs -tail=20 airflow-webserver` to capture one error line for your table.
- (d) Restore Postgres **before** restarting other services if anything else crashed:

```
docker compose start postgres
```

Wait until `docker compose ps` shows Postgres healthy again, then refresh the UI.

Scenario (summary)	Your prediction	Actual behaviour
A: Webserver stopped; CLI trigger	...	...
B: Scheduler stopped; UI trigger	...	...
C: Postgres stopped; reload UI	...	...

Info

After each scenario, confirm `docker compose ps` before moving on. If a service fails to restart, run `docker compose up -d` once from `lab3_student/`.

Q0.5 – Resource control with Pools

Airflow **Pools** let you cap the number of tasks of a given category that can run concurrently. They are the standard production technique for protecting a fragile downstream system (e.g. a partner API rate-limited at 5 requests/s) from being hammered by Airflow.

- (a) Open Admin → Pools in the UI. Create a new pool named `lab_pool` with **2 slots**.
- (b) Open the existing `example_hello` DAG. Without changing its logic, be ready to explain to the instructor how you would force its tasks to use this pool (no need to actually modify it).
- (c) Now imagine 50 daily DAGs all hitting the same partner API, each with 4 tasks using `lab_pool`. How many tasks run simultaneously? Why is this a much better protection than relying on the worker count alone?

STOP — call the instructor (CP1: Setup & Architecture)

Target time: ~30 min after you start the lab clock.

Before you call: check that everything you need to answer the questions is ready.

Rubric:

CP1.1: Airflow UI loads at `localhost:8080`; you are logged in.

CP1.2: `docker compose ps` shows all containers *Up* or *healthy*.

CP1.3: Q0.2 table is correct: all six containers mapped to their CM3 roles with evidence.

CP1.4: Q0.3: you can name the four metadata tables (`dag`, `task_instance`, `dag_run`, and `connection` or `variable`).

CP1.5: Q0.4: you explain why stopping the scheduler does *not* prevent a manual trigger from the UI, and what the webserver does vs the scheduler.

CP1.6: Q0.5: pool `lab_pool` with **2 slots** exists in Admin → Pools; you can explain why pools beat raw worker count for throttling a fragile downstream.

Part 1: First DAGs – Operators, TaskFlow, and Dependencies**Info**

A reminder from the CM3: a DAG is just a Python script. The scheduler reads every file in the `dags/` folder and registers DAGs it finds there: classic DAG objects, or TaskFlow DAGs built with `@dag` where the decorated factory is **called once** at import time (e.g. the trailing `example_hello()` line). You will write your files in `dags/`; they appear in the UI within ~30 seconds, no restart needed.

Q1.1 – The hello DAG (warm-up)

An example DAG `dags/example_hello.py` is provided. Open it, read it, and answer:

- How many tasks does it define? What are their Airflow task ids in the UI?
- The DAG uses the **TaskFlow API**: `@dag` on the outer factory and `@task` on the inner functions. What concrete work does each task perform (what appears in the task logs)?
- The body stores the two tasks in variables `hello` and `goodbye`. How does `goodbye = say_goodbye(hello)` encode both the run order and the passing of the first task's return value into `say_goodbye`? What does the extra line `hello » goodbye` mean — is `»` normal integer bitshift in Python, or an Airflow-specific composition operator? Why is it redundant here yet useful to see before `q1_pipeline`?
- Trigger the DAG from the UI. Watch the Grid view. In what order do the tasks turn green?

Q1.2 – Your first DAG: BashOperator

Create a new file `dags/q1_pipeline.py`. Inside, build a DAG with id `q1_pipeline` that contains **four tasks**:

- `print_date` – prints the current date with `date`.
- `whoami` – prints the user with `whoami`.
- `count_dags` – counts how many `.py` files are in `/opt/airflow/dags`.
- `done` – prints "Lab 3 – Part 1 complete".

Dependencies: `print_date` runs first, then `whoami` and `count_dags` run in **parallel**, and finally `done` runs after both.

Hint

The list shorthand works: `a » [b, c] » d`. See the Airflow docs on Bitshift composition operators if unsure.

Q1.3 – Same DAG, TaskFlow API

Rewrite the exact same logic in a new file `dags/q1_taskflow.py` using the modern `@dag` and `@task` decorators (no `BashOperator`). All four steps must become **Python functions** returning strings.

- Show in the UI that both DAGs produce the same shape (Graph view).
- Compare: which version do you find more readable? Which is more powerful in your opinion? Justify with one concrete example.

Q1.4 – Diamond DAG

Create `dags/q1_diamond.py`, this time using a **mix** of one `BashOperator` and three Python tasks (`@task` decorator):

Task id	Type	Body
<code>extract</code>	Bash	<code>echo "extracting..." && sleep 5</code>
<code>transform_a</code>	Python	sleep 3 seconds, return string "A done"
<code>transform_b</code>	Python	sleep 7 seconds, return string "B done"
<code>load</code>	Python	print both return values, then print "loaded"

Wire them so `transform_a` and `transform_b` run in parallel after `extract`, and `load` waits for both.

Q1.5 – Observability tour

Trigger `q1_diamond` once and answer:

- In the **Grid** view, what is the total wall-clock time of the DAG run? Is it $\geq 5 + 3 + 7 = 15$ s or closer to $5 + \max(3, 7) = 12$ s? Why?
- Click on the `transform_b` task and find the **logs**. Paste the line showing the start of execution.
- In the **Graph** view, what color encodes a successful task? A failed one? A skipped one?
- How did `load` get the return values of `transform_a` and `transform_b`? In which database table are those values stored? (Hint: you saw it in Q0.3 – and it starts with `xcom`.)

Q1.6 – The parsing trap

Open `dags/q1_diamond.py` and add this line at the **top** of the file, *outside* any function or DAG context:

```
import time
print(f"[BAD] Parsing q1_diamond at {time.strftime('%H:%M:%S')}")
```

Save the file. Wait 2 minutes. Airflow logs DAG parsing under `/opt/airflow/logs/` inside the containers; that folder is **bind-mounted** to `logs/` next to your `docker-compose.yaml` on the host. From the `lab3_student/` directory, search those files for your marker (the exact path names may vary; a recursive grep is enough):

```
grep -R "[BAD]" logs/
```

Answer:

- How many times did the `[BAD]` line print in 2 minutes?

- (b) What does this tell you about the cost of any code placed at the top level of a DAG file?
- (c) Why is this an anti-pattern? (Reference the CM3.) Remove the line before continuing.

Q1.7 – TaskGroups: visual organization

Take a copy of `q1_diamond.py` into `dags/q1_grouped.py`. Wrap the two transforms inside a **TaskGroup** named `transforms`. The DAG body should look roughly like:

```
from airflow.decorators import task_group

@task_group(group_id="transforms")
def transforms():
    return [transform_a(), transform_b()]

extract >> transforms() >>> load
```

Answer:

- (a) Compare the **Graph** view of `q1_diamond` and `q1_grouped`. What changes visually?
- (b) Inspect the full task ids in the UI of `q1_grouped`. What prefix appears on `transform_a` and `transform_b`?
- (c) Sketch in one sentence when a TaskGroup becomes worth the extra code: is it for 4 tasks, 40 tasks, or 400 tasks?

STOP — call the instructor (CP2: First DAGs)

Target time: ~1 h 30 after lab start.

Before you call: check that everything you need to answer the questions is ready.

Rubric:

CP2.1: `q1_pipeline`: at least one successful run (four BashOperators; fan-out then join).

CP2.2: `q1_taskflow`: at least one successful run (same shape, TaskFlow API).

CP2.3: `q1_diamond`: at least one successful run; wall-clock ≈ 12 s (parallel transforms, not $5+3+7$).

CP2.4: You explain how XCom carried the return values from `transform_a` / `transform_b` into `load` (and which metadata table stores XComs).

CP2.5: Q1.6: you show scheduler logs where [BAD] printed at least four times in two minutes (parsing-trap evidence).

CP2.6: Q1.7: `q1_grouped` Graph view with the `transforms` TaskGroup shown.

Part 2: Sensors, Ingestion, and Validation

In production, your pipeline does **not** control when the input file arrives. A vendor, an upstream team, or a partner API delivers it – often late. Your job is to *wait correctly*.

Q2.1 – Simulate the vendor

A helper script `scripts/vendor_drop.py` simulates the upstream system dropping a CSV in the watch folder `data/incoming/`. From your host machine, run:

```
python scripts/vendor_drop.py --date 2026-05-19 --delay 0
```


Inspect the file it created (a tiny CSV of ~200 fake transactions). What is the file naming convention? You will need it for the sensor.

Q2.2 – A FileSensor that waits

Create `dags/q2_ingest.py`. The DAG must:

1. Use a `FileSensor` that waits for the day's CSV at `/opt/airflow/data/incoming/transactions_{{ ds }}.csv` on disk. In Airflow 2.4+, `FileSensor.filepath` is **relative** to the File (path) connection `fs_default`; the lab `docker-compose.yaml` sets that base to `/opt/airflow/data`, so use `filepath="incoming/transactions_{{ ds }}.csv"`.
2. Set the sensor in **reschedule** mode with a 30-second poke interval and a timeout of 10 minutes.
3. When the file appears, a Python task `ingest` reads it with DuckDB, prints the row count, and writes the data to `/opt/airflow/data/raw/transactions_{{ ds }}.parquet`.

Hint

The Jinja template `{{ ds }}` expands to the execution date in YYYY-MM-DD format. You can use it inside templated operator fields (`FileSensor`, `BashOperator`, ...).

TaskFlow caveat: in a `@task` function, a parameter named `ds` is filled from the Airflow context. Its default must be `None` only (e.g. `def ingest(ds: str | None = None)`). Do *not* write `ds="{{ ds }}"` as a Python default; Airflow 2.10 rejects it.

Helper functions are provided in `include/duck.py`. You may simply call `from include.duck import csv_to_parquet` from your DAG.

Q2.3 – Validate before trusting

Add a third task `validate` that runs after `ingest`. It must:

- Open the Parquet file produced by `ingest`.
- Compute the row count and the sum of the `amount_eur` column.
- **Fail loudly** (raise `AirflowFailException`) if:
 - Row count is below 50, or
 - Sum of amounts is exactly 0 (this would indicate corrupt input).
- Otherwise, print a one-line summary.

Hint

Classic operator + TaskFlow: do *not* wire `wait_for_file` » `validate(ingest())`. Python calls `ingest()` first when building the graph, so you only get `ingest` → `validate` and `wait_for_file` → `validate`: the sensor is **not** upstream of `ingest`, which then runs immediately and crashes if the CSV is missing. Use instead `ingested = ingest(); wait_for_file` » `ingested; validate(ingested)` (same idea for `PythonSensor` in Q2.7).

Q2.4 – Watch it wait, then run

Make sure no file exists in `data/incoming/` for today's date yet. Trigger `q2_ingest` with execution date 2026-05-20.

The DAG should enter the `FileSensor` and **stay yellow** (up for reschedule).

In a second terminal, after ~1 minute, run:

```
python scripts/vendor_drop.py --date 2026-05-20 --delay 0
```

Within 30s the sensor should detect the file. Verify the rest of the DAG runs to green.

Q2.5 – Poke vs reschedule

Open `q2_ingest.py` again. Change the sensor mode to `"poke"` and re-trigger.

Answer:

- In `poke` mode, what is the state of the worker slot while the sensor waits? (Use the UI **Pools** or run `docker compose exec airflow-worker celery inspect active` to inspect.)
- Suppose you have 16 worker slots and 200 daily DAGs that each start with a 4-hour `FileSensor`. What is the maximum number of those DAGs that can wait simultaneously in `poke` mode? In `reschedule` mode?
- Restore the mode to `reschedule`.

Q2.6 – The corrupt-file scenario

Trigger `vendor_drop.py` with the special `-corrupt` flag for date 2026-05-19. This produces a file with all amounts set to zero.

```
python scripts/vendor_drop.py --date 2026-05-19 --corrupt
```

Hint

To run a DAG with a past execution date, use the `-exec-date` flag:

```
docker compose exec airflow-scheduler airflow dags trigger q2_ingest -e 2026-05-11
T00:00:00+00:00
```

Now trigger `q2_ingest` for date 2026-05-19. What happens? Which task fails? What is the exact exception message? Keep the failing task log open in your browser – the instructor will inspect it during CP3.

Q2.7 – Stable sensors: not just existence

A `FileSensor` returns success the moment a file exists. In production, this is a known source of bugs: if the vendor uploads via FTP/rsync, the file may exist *while still being written*. Your downstream task then reads a half-baked CSV and either crashes or, worse, silently truncates rows.

Use the `-slow` flag of the vendor simulator to reproduce the issue:

```
python scripts/vendor_drop.py --date 2026-05-20 --slow
```

This writes the CSV in **three chunks separated by 10 s pauses**, simulating a slow upload.

Replace the `FileSensor` in a *new* DAG `dags/q2_stable.py` with an `@task.sensor` (TaskFlow sensor) or a `PythonSensor` that returns `True` only when **both** conditions hold:

- The file exists **and** its size is at least 200 bytes.
- The file's size has **not changed** between two polls 5 s apart (i.e. the writer has stopped extending it).

Trigger `q2_stable` for 2026-05-20, then launch `vendor_drop.py -slow` for the same date. Observe in the sensor log how it sees the file growing and refuses to succeed until it stabilizes.

- Paste two consecutive log lines from your sensor showing the file growing.
- Paste the line where the sensor finally returns success.
- In which production setup (real-world example) would a size-stability check be insufficient? Hint: think about files that contain trailing metadata or write-then-truncate patterns.

STOP — call the instructor (CP3: Sensors & Ingestion)

Target time: ~2h 15 after lab start.

Before you call: check that everything you need to answer the questions is ready.

Rubric:

CP3.1: q2_ingest for 2026-05-19: FileSensor was up_for_reschedule, then turned green after the file drop.

CP3.2: Parquet exists at data/raw/transactions_2026-05-19.parquet (host path) or the equivalent under /opt/airflow/data/raw/ inside the worker.

CP3.3: q2_ingest for 2026-05-18 (corrupt): validate fails with a clear message; you show the task log to the instructor.

CP3.4: Q2.5(b): you articulate how poke vs reschedule consumes worker slots, including the “16 slots, 200 DAGs” scenario.

CP3.5: Q2.7: q2_stable for 2026-05-23 survives vendor_drop.py -slow; sensor logs show “file growing” then eventual success.

Part 3: Scheduling, Idempotence, and Backfill

This is the heart of the lab. By the end of this part, your pipeline will be **safely re-runnable across any date range**, which is the property that makes Airflow worth using in the first place.

Q3.1 – Schedule a daily DAG

Copy q2_ingest.py into dags/q3_daily.py. Modify the DAG id to q3_daily and configure:

- `schedule="@daily"`
- `start_date = datetime(2026, 5, 10)`
- `catchup = False` for now

Unpause the DAG. Don't trigger anything manually. Observe what happens over ~2 minutes.

- Does Airflow start running anything? Why or why not?
- Look at the **Next Run** column in the DAG list. What date does it show? Why is it not today's date?
- Open the DAG details. What is `data_interval_start` and `data_interval_end` for the next scheduled run?

Info

This is the CM3 `execution_date` concept made concrete. A run for “May 19” fires on May 20 at midnight, because the May 19 data interval is only complete at the end of May 19.

Q3.2 – The first idempotence test

Generate ten days of historical data in one go:

```
python scripts/vendor_drop.py --range 2026-05-10:2026-05-19
```

Now enable catchup:

- Set `catchup=True` in the DAG.
- Save the file.

Now delete the DAG from the UI. Wait ~30 seconds to see the DAG showing up again, and unpause it. Watch the Grid view fill in. Airflow will schedule **one run per missed day**. Verify that 10 DAG runs eventually appear (some will be in parallel up to the limit of `max_active_runs`).

- How many Parquet files do you see under `data/raw/`? List them.
- How big is each one? (Use `ls -la data/raw/` or PowerShell equivalent.)
- All runs are green. Inspect one Parquet file with DuckDB:

```
docker compose exec airflow-worker python -c "import duckdb; print(duckdb.read_parquet('/opt/airflow/data/raw/transactions_2026-05-15.parquet').count('*').fetchone())"
```

Record the row count for May 15.

Q3.3 – Break idempotence on purpose

Now you will write a **non-idempotent** version of the ingestion to feel the pain.

Create `dags/q3_evil.py`. It must do the same job as `q3_daily`, with two deliberate flaws:

- The output file path uses `datetime.now()` instead of `{{ ds }}`.
- The ingest task appends to a single growing file `data/raw/all_transactions.parquet` instead of writing per-day Parquet.

Hint

DuckDB supports `COPY ... TO 'file.parquet' (FORMAT PARQUET, APPEND)` in recent versions, or you can read the existing file, union the new rows, and overwrite.

Run a 5-day backfill for `q3_evil`:

```
docker compose exec airflow-scheduler airflow dags backfill q3_evil -s 2026-05-12 -e 2026-05-16 --reset-dagruns
```

Now **re-run the same backfill a second time** with the same command.

Answer:

- Did Airflow re-execute the runs? (Hint: `--reset-dagruns` forces it.)
- Inspect the row count of `all_transactions.parquet` before and after the second run. Are the numbers the same or doubled?
- Which of the two flaws ((a) or (b) above) caused the duplication? Could either flaw be triggered by an automatic Airflow retry on transient failure?

Q3.4 – Prove idempotence on `q3_daily`

Repeat the same double-backfill experiment, but this time on `q3_daily`:

```
docker compose exec airflow-scheduler airflow dags backfill q3_daily -s 2026-05-12 -e 2026-05-16 --reset-dagruns
```

- Compute the SHA-256 of `data/raw/transactions_2026-05-15.parquet` before and after the second backfill. They should be identical.
- Explain in one sentence which design choices made `q3_daily` idempotent.

Hint

On Windows PowerShell: `Get-FileHash data\raw\transactions_2026-05-15.parquet -Algorithm SHA256`.

On Mac/Linux: `sha256sum data/raw/transactions_2026-05-15.parquet`.

Q3.5 – Aggregate by day

Add a downstream task `aggregate` to `q3_daily` that computes daily revenue summary statistics with DuckDB:

- Per product category: total revenue (`SUM(amount_eur)`) as `revenue`.
- Per product category: transaction count (`COUNT(*)`), aliased e.g. as `tx_count`.
- Sort rows by `revenue` descending so the best-selling categories appear first (for a single day, the first three rows are the top-3 categories by revenue).

Write the result as Parquet to `data/agg/daily_summary_{{ ds }}.parquet`.

You should use the `daily_aggregate()` function from the `include/duck.py` file.

Re-run the backfill. Then in a single DuckDB query, read all aggregated files at once (use the `*` glob on `daily_summary_*.parquet`) and produce a result that covers the calendar days 2026-05-10 through 2026-05-19 (attach each file's logical date from the path or filename if helpful). Be ready to show the query and its output at CP4.

Hint

Reference SQL (exemple of query accepted):

```
SELECT
  category,
  SUM(revenue) AS total_revenue_eur,
  SUM(tx_count) AS total_transactions
FROM read_parquet(
  '/opt/airflow/data/agg/daily_summary_2026-05-1*.parquet'
)
GROUP BY category
ORDER BY total_revenue_eur DESC
LIMIT 3;
```

Q3.6 – Catchup vs Backfill (analysis)

- What is the difference between setting `catchup=True` on a DAG and running `airflow dags backfill`? Name one scenario where you would use each.
- In the CM3 you saw the danger of `catchup=True` with a year-old start date. How many DAG runs would be scheduled for a daily DAG with `start_date = 2020-01-01` and `catchup=True`, deployed today?
- What is `max_active_runs` and how does it protect you in that scenario?

Q3.7 – The runaway catchup experiment

You will now reproduce – safely – the disaster scenario the CM3 warns against.

Create `dags/q3_old_dag.py` with these exact parameters:

```
start_date = datetime(2024, 1, 1)
schedule = "@daily"
catchup = True
max_active_runs = 2
```

The DAG body itself can be a single `BashOperator` running `sleep 20 && echo "ran for {{ ds }}"` – the goal is purely to observe the scheduler's behaviour.

Without `max_active_runs`, this DAG would request the scheduler to spin up hundreds of runs at once. With `max_active_runs=2`, the scheduler queues them and only releases 2 at a time.

Procedure:

1. Save the file. Wait ~45s for the scheduler to pick it up.

2. Unpause the DAG in the UI.
3. Watch the Grid view for **few minutes**.
4. Pause the DAG.

Answer:

- (a) How many DAG runs are in state **running** at any given moment?
- (b) How many DAG runs are in state **queued** after 3 minutes?
- (c) Query the metadata DB: how many `dag_run` rows exist for `dag_id = 'q3_old_dag'`? (Use the `psql` command from Q0.3.)
- (d) Without `max_active_runs=2`, what would have happened? Be specific about what would have suffered first: the worker, the scheduler, the DB, or the disk.

Important

After the experiment pause the workflow in the UI and then run `docker compose exec airflow-scheduler airflow dags delete q3_old_dag -yes` to remove the queued runs from the metadata DB before continuing.

STOP — call the instructor (CP4: Scheduling, Backfill, Idempotence)

Target time: ~end of the lab.

Before you call: check that everything you need to answer the questions is ready.

Rubric:

CP4.1: `q3_daily`: 10 runs green in Grid view (dates 2026-05-10 through 2026-05-19).

CP4.2: Ten per-day Parquet files under `data/raw/`, correctly named.

CP4.3: Q3.3: `q3_evil` — proof that the consolidated file or row count *grew* after a second identical backfill.

CP4.4: Q3.4: two SHA-256 hashes for the same file before and after a second backfill — they must match (idempotence proof).

CP4.5: Q3.5: DuckDB query over May 10–19 aggregations: per-category `SUM(revenue)` and `SUM(tx_count)`, query + output shown.

CP4.6: Q3.6: correct verbal distinction between `catchup=True` and `airflow dags backfill`, plus one use case for each.

CP4.7: Q3.7: `q3_old_dag` — only two runs **running** at a time; queued runs visible; you can state what would break first without `max_active_runs`.

Part 4: Anti-patterns Hunt & Resilience

The repository contains a DAG file `dags/broken_dag.py` which a previous intern wrote in a hurry before going on holiday. It runs. It even produces output. But it is a **textbook collection of anti-patterns**, and in two weeks it has already caused three production incidents.

Q4.1 – The Five Crimes

Read `dags/broken_dag.py` carefully. It contains **exactly five** anti-patterns from the CM3. For each one, fill in the table:

#	Anti-pattern (CM3 name)	(CM3)	Line(s) in the file & code snippet	Why it is dangerous
1	...		...	...
2	...		...	...
3	...		...	...
4	...		...	...
5	...		...	...

Hint

The CM3 lists 5 common pitfalls (#1 to #5). The 5 crimes in `broken_dag.py` map one-to-one to those. If you find only 4, look at top-level code again.

Q4.2 – The Fix

Create `dags/q4_fixed.py` that does the same business logic as `broken_dag` but fixes all five anti-patterns.

Q4.3 – Watch the retries in action

Open `include/transform.py` and locate the function used by `q4_fixed`. Add a temporary line that raises an exception on **the first call** and succeeds afterwards:

```
import os, pathlib
_marker = pathlib.Path("/tmp/q4_failed_once")
if not _marker.exists():
    _marker.touch()
    raise RuntimeError("Simulated transient failure (will succeed on retry)")
```

Trigger `q4_fixed` for any date. Watch the task *fail once*, then **auto-retry** and succeed.

In the UI:

- Open the task instance details. How many attempts are recorded?
- Find the log for attempt 1. Paste the line with the `RuntimeError`.
- Find the log for attempt 2. Paste the line showing the success.
- Remove the marker file and the test line afterwards:

```
docker compose exec airflow-worker rm /tmp/q4_failed_once
```

Q4.4 – SLA miss

Edit `q4_fixed` so that the slow task has an SLA of 20s (`sla=timedelta(seconds=20)`), then add a deliberate `time.sleep(30)` at the start of that task.

Trigger the DAG and wait for it to finish. Then in the UI:

- Navigate to Browse → SLA Misses. How many entries are there?
- What information does the SLA miss record contain?
- In a production environment, where would this notification typically be routed? (Hint: not just the UI.)
- Remove the artificial sleep and restore a realistic SLA value before continuing.

Q4.5 – The XCom temptation

In `q4_fixed`, you currently pass file paths between tasks via XCom. Imagine that instead, you decided to pass the entire DuckDB result table (a Pandas DataFrame of 500 k rows) through XCom.

- (a) Without writing the code, estimate the size in MB of the XCom payload.
- (b) Which Airflow component would suffer first?
- (c) Read the Airflow docs on **XCom backends**. Name one production-grade backend that can store large XComs outside Postgres.
- (d) Is using such a backend a substitute for “don’t put data in XCom”? Argue both sides in 2–3 sentences.

Q4.6 – Callbacks: simulate a production observability stack

In the CM3 you saw that real teams route failures to Slack, PagerDuty, or Opsgenie. Airflow exposes hooks for this through **callbacks**: arbitrary Python functions called when a task or DAG transitions to a final state.

Extend `q4_fixed.py` as follows:

1. Define a function `notify_failure(context)` that writes a JSON line (one per task failure) to `/opt/airflow/data/incidents.jsonl`. Each line must contain: `timestamp`, `dag_id`, `task_id`, `run_id`, `exception_type`, `exception_message`.
2. Define a function `notify_retry(context)` that writes a single line `<task_id> retried (attempt N)` to the same file.
3. Wire both into the DAG’s `default_args` as `on_failure_callback` and `on_retry_callback`.

Test it by reintroducing the marker-file failure from Q4.3 (the file should now record both a retry event *and*, if you remove the marker file logic to make the failure permanent, a final failure event).

Answer:

- (a) Paste the contents of `incidents.jsonl` after the experiment.
- (b) Compare callbacks with the `email_on_failure` flag from earlier: what can a callback do that an email flag cannot?
- (c) In production, where would `notify_failure` typically send the record instead of a local file? Name two realistic destinations.

Info

The `context` dict passed to a callback contains `task_instance`, `dag_run`, `exception`, and the standard templated variables. See the Airflow docs on *Callbacks and Notifiers* for the full list.

STOP — call the instructor (CP5: Anti-patterns & Resilience)

Target time: You will probably can't get there before the end of the lab. If you do, congratulations!

Before you call: check that everything you need to answer the questions is ready.

Rubric:

CP5.1: Q4.1: all five anti-patterns from `broken_dag.py` identified, matching the five categories listed at the start of Part 4 (with line references in your table).

CP5.2: `q4_fixed`: at least one full successful run end-to-end (all tasks green).

CP5.3: Q4.3: retry observed — first attempt fails with the injected `RuntimeError`, second attempt succeeds; both log excerpts visible.

CP5.4: Q4.4: at least one SLA miss visible under Browse → SLA Misses from the artificial 20 s SLA / 30 s sleep experiment.

CP5.5: Q4.5: you can articulate the XCom size problem and what an external XCom backend does (and does not) solve.

CP5.6: Q4.6: `/opt/airflow/data/incidents.jsonl` (or host `data/incidents.jsonl`) contains at least one *retry* event and one *failure* event; you can explain callback vs `email_on_failure` if asked.

Part 5: Modern Patterns – TaskGroups, Branching, Datasets

Info

This part has **no checkpoint** and is **not part of the main grade**. Each question here introduces a feature shipped with Airflow 2.4+ that you would encounter in any respectable production code base in 2026. Pick the ones that interest you most. If you have outstanding work to show before the lab ends, flag the instructor; they may grant you some bonus points on the final mark.

Q5.1 – TaskGroups inside the pipeline

Apply the TaskGroup pattern you learned in Q1.7 to `q4_fixed.py`. Group the tasks into two named blocks visible in the Graph view:

- **ingestion**: contains `ingest` and `validate`.
- **processing**: contains `transform` (and `aggregate` if you wrote one).

Show a screenshot of the Graph view with both groups collapsed, then expanded.

Q5.2 – Branching: skip the cheap days

Not every day is worth the full pipeline. If the input has fewer than 100 transactions (rare, but it happens on long weekends), the aggregate is meaningless. You will add a **branch** that routes the DAG either to the full transform or to a cheap “skip” task that logs the decision and exits cleanly.

Create `dags/q5_branching.py` – a TaskFlow DAG with the following tasks:

- `ingest` – reuse `include.duck.csv_to_parquet`.
- `decide` – decorated with `@task.branch`; based on the `row_count` returned by `ingest`, it returns either the string `"transform"` or `"skip_small_day"`.
- `transform` – the heavy logic.
- `skip_small_day` – just logs “Skipping: row_count=<n>”.
- `notify_done` – final task; runs whichever branch was taken. Use `trigger_rule="none_failed_min_one_success"`.

Test it with two scenarios:

```
python scripts/vendor_drop.py --date 2026-05-24 --rows 30 # tiny day
python scripts/vendor_drop.py --date 2026-05-25 --rows 400 # normal day
```

Trigger `q5_branching` once per date and answer:

- On the 30-row day, what is the state (success, skipped, or failed) of `transform`? Of `skip_small_day`?
- Without the `trigger_rule` on `notify_done`, what state would it end up in? Why? (Hint: think about which upstream tasks were skipped.)
- Name one real-world scenario where you would want branching but `ShortCircuitOperator` would be a simpler alternative.

Q5.3 – Datasets: event-driven scheduling

So far, every downstream DAG has been triggered *by time*. Airflow 2.4+ adds **Datasets** (a.k.a. Data-Aware Scheduling): a DAG runs automatically whenever another DAG writes to a declared dataset.

Producer side. Modify `q3_daily.py`:

```
from airflow.datasets import Dataset

DAILY_AGG = Dataset("file:///opt/airflow/data/agg/daily_summary.parquet")

# On the aggregate task only:
@task(outlets=[DAILY_AGG])
```

```
def aggregate(...):
    ...
```

Consumer side. Create `dags/q5_dashboard_refresh.py`:

```
from airflow.datasets import Dataset

DAILY_AGG = Dataset("file:///opt/airflow/data/agg/daily_summary.parquet")

@dag(
    dag_id="q5_dashboard_refresh",
    start_date=datetime(2026, 5, 10),
    schedule=[DAILY_AGG], # <-- not a cron expression!
    catchup=False,
)
def q5_dashboard_refresh():
    @task
    def refresh():
        print("Dashboard refreshed.")
    refresh()

q5_dashboard_refresh()
```

Open the **Datasets** view in the UI: <http://localhost:8080/datasets>.

- Trigger `q3_daily` once. Does `q5_dashboard_refresh` fire automatically? Within how many seconds?
- Take a screenshot of the Datasets graph (producers → datasets → consumers).
- Compare event-driven Dataset scheduling to using an `ExternalTaskSensor`. Name one advantage and one disadvantage of each.
- Could a Dataset be triggered *multiple times* during the same logical day if its producer DAG runs twice (e.g. a manual rerun)? Why is this important to know?

Q5.4 – Reflection: modern features against classic problems

Match each modern feature on the left to the classic CM3 problem it solves better:

Modern feature	Classic problem it improves on
Datasets	...
<code>@task.branch</code>	...
TaskGroups	...
<code>on_failure_callback</code>	...
Pools	...

Part 6: Bonus – For Very Fast Students

These bonus tasks are independent and you can do them in any order. Show your work to the instructor live; they may grant you some bonus points on the final mark.

B1 – Dynamic mapping with `.expand()`

Modern Airflow (2.3+) supports dynamic task mapping. Modify `q3_daily` so that the `ingest` step automatically maps over **all CSV files** present in `data/incoming/` for the execution date (in case the vendor split the daily batch into multiple files).

Read the docs on `@task.expand` and `XComArg`. Demonstrate the result on a day where you generated 3 split files using:

```
python scripts/vendor_drop.py --date 2026-05-20 --split 3
```

B2 – Airflow Variables and Connections

Currently the input/output paths are hardcoded in the DAGs. Move them to:

- **Variables** for the paths (see Admin → Variables in the UI).
- A **Connection** named `duckdb_default` pointing to your local DuckDB database file (use a generic connection with the path in the `host` field).

Modify one DAG to read both. What is the practical advantage over hardcoding?

B3 – TriggerRules: an error-reporting branch

Add to `q4_fixed` a final task `notify_failure` configured with `trigger_rule="one_failed"`. It must print a message to the logs whenever *any upstream task* of the DAG fails.

Test it by deliberately failing one task. What is the state of `notify_failure` when everything succeeds? What is its state when one task fails?

List three other trigger rules and describe a use case for each.

B4 – Unit-test your DAG

Create a file `tests/test_dags.py` with at least three checks:

1. All DAG files in `dags/` import without error (`DagBag().import_errors == {}`).
2. `q4_fixed` has **at least 4 tasks**.
3. Every task in `q4_fixed` has `retries >= 1`.

Run the tests with:

```
docker compose exec airflow-worker pytest /opt/airflow/tests/test_dags.py -v
```

Why would a real team run these tests in CI *before* merging a DAG change?